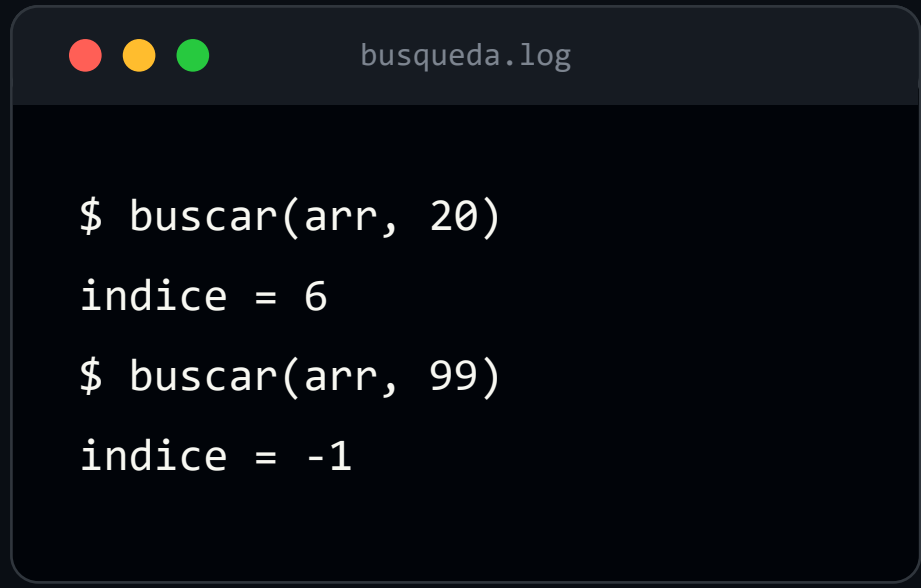
CE-2103 · Lección 04

Algoritmos de Búsqueda

Encontrar un valor en un array y un camino en un mapa



Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

¿Qué aprenderemos hoy?

1. Aplicar la búsqueda secuencial y reconocer su costo `O(n)`
2. Explicar la búsqueda binaria, por interpolación y por salto
3. Comparar la complejidad de los algoritmos de búsqueda en arrays
4. Describir el pathfinding básico y cómo rodear obstáculos
5. Modelar un mapa como grafo y aplicar BFS, Dijkstra y A* (con G, H y F)

// búsqueda secuencial

Búsqueda secuencial (lineal)

La solución trivial: revisar cada elemento hasta encontrar el buscado.

Comparar

En el peor caso, cada elemento se compara contra la llave de búsqueda.

Si hay match

La búsqueda termina y se retorna el índice del elemento.

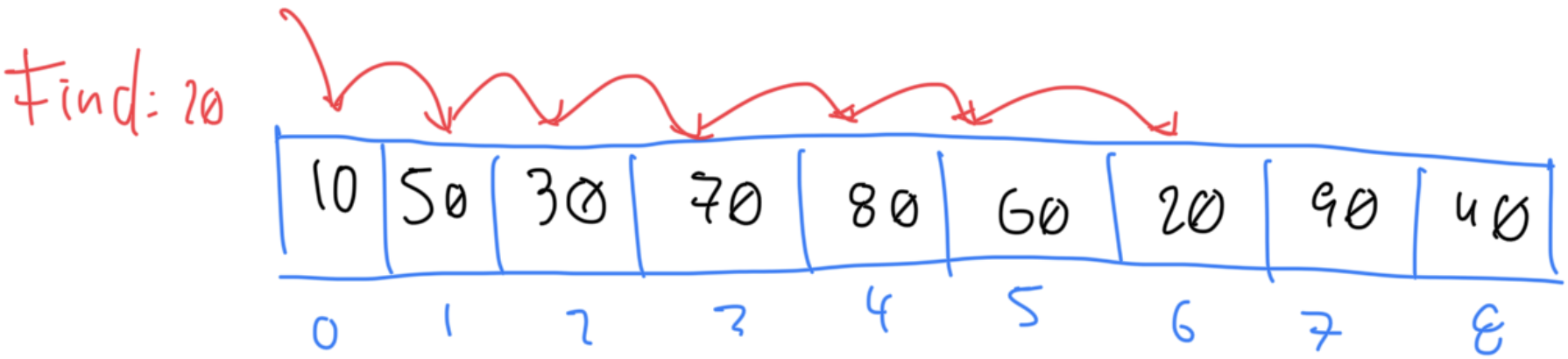
Si no hay match

Se retorna `-1` (u otro índice negativo).

Importante: si la colección está **desordenada**, es la única opción.

// búsqueda secuencial

Ejemplo: buscar el 20



Se comparan 7 elementos (índices 0 a 6) hasta encontrar el `20` en el índice 6.

// búsqueda secuencial

Implementación en Java

SequentialSearch.java

```java
public class SequentialSearch {
    public static int search(int[] arr, int x) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == x) {
                return i;
            }
        }
        return -1;
    }
}
```

Mejor caso

El elemento es el primero.

O(1)

Peor caso

Recorre el array completo.

O(n)

Ojo: es **ineficiente** para colecciones grandes.

Pero: si el array no está ordenado (o no se quiere ordenar), es la única opción.

// búsqueda binaria

Búsqueda binaria: dividir el arreglo

1

Comparar

Se compara la llave con el elemento del medio.

2

Descartar

Si es menor, se sigue a la izquierda; si es mayor, a la derecha.

3

Repetir

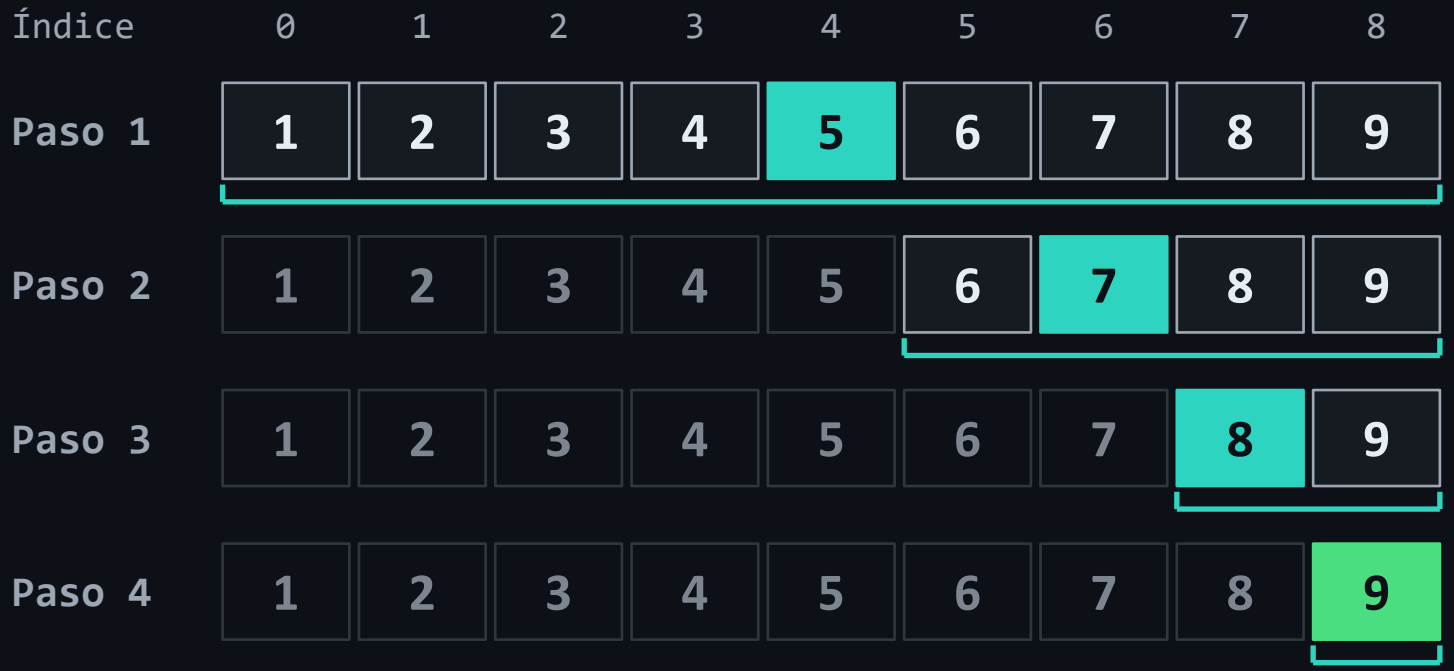
Hasta hallar el valor o hasta que el subarreglo quede vacío.

Requisito: el arreglo debe estar **ordenado**. A diferencia de la secuencial, no lo recorre de extremo a extremo.

// búsqueda binaria

Ejecución: buscar el 9

Buscar: 9 **Elemento medio** **Encontrado** **Corchete: rango de búsqueda**



`left=0 right=8 mid=4`
5 < 9 → mitad derecha

`left=5 right=8 mid=6`
7 < 9 → mitad derecha

`left=7 right=8 mid=7`
8 < 9 → mitad derecha

`left=8 right=8 mid=8`
9 == 9 → encontrado

Ahorro: 4 comparaciones en 9 elementos; la secuencial habría hecho 9.

// búsqueda binaria

Implementación en Java

binarySearch

```java
public static int binarySearch(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Descartar mitades

Si `arr[mid]` es menor que `target`, sube `left`; si no, baja `right`.

Costo

Cada paso descarta la mitad del rango.

O(log n)

// interpolación

Binaria vs. interpolación

Búsqueda binaria

- Compara siempre contra el elemento **central**
- Divide el rango en dos mitades
- No usa el valor de la llave para elegir la posición

vs

Interpolación

- Considera la **llave** antes de elegir contra cuál elemento comparar
- Usa los valores en los extremos del array
- Mejora si los valores están **distribuidos uniformemente**

Analogía: para buscar «Bob» en un directorio telefónico no se abre por la mitad, sino **cerca del principio**. Eso hace la interpolación: es una **mejora** de la binaria que solo cambia cómo se calcula `mid`.

// interpolación

La posición estimada

Cálculo de la posición del elemento medio

```
mid = low + ((x - arr[low]) * (high - low)) / (arr[high] - arr[low])
```

Qué cambia

Es el mismo código de la binaria; solo se calcula distinto `mid`.

Variables

`low` y `high` son índices; `arr[low]` y `arr[high]`, los valores en esos índices.

Se adapta a la llave

Si `x` está cerca de `arr[high]`, `mid` cae cerca de `high`.

Complejidad: `O(log log n)` en el caso promedio (distribución uniforme); `O(n)` si los datos no son uniformes.

// interpolación

Interpolación: un ejemplo con números

Buscar: 70 | Comparado | Encontrado | Corchete: rango de búsqueda



`0 + ((70-10)*(9-0)) / (100-10) = 6`
arr[6] = 70 → 1 comparación

índices comparados: `4, 7, 5, 6`
4 comparaciones

Idea clave: el `70` está a **2/3** del rango de valores (10 a 100), así que `mid` cae a 2/3 del rango de índices (0 a 9): el **6**.

Ojo: con valores no uniformes, como `[5, 6, 9, 11, 15, 18, 20, 25, 28, 39]`, la ventaja desaparece: para buscar el 28, ambas hacen 3 comparaciones.

// búsqueda por salto

Búsqueda por salto (Jump Search)

Array ordenado

Aplicable solo a arrays ordenados.

Saltos de √n

Avanza en bloques de tamaño √n, un número fijo de elementos por salto.

Tamaño del bloque

Determinarlo bien es crucial para el rendimiento del algoritmo.

Complejidad temporal: `O(sqrt(n))`. Compara menos elementos que la búsqueda secuencial.

// búsqueda por salto

Jump Search: ejemplo con n = 16



n = 16 y √n = 4: se salta de 4 en 4 hasta pasar el valor (144 > 55) y se retrocede.

Comparación de los algoritmos

Algoritmo	¿Requiere orden?	Complejidad temporal	Idea clave
Secuencial	No	O(n) peor · O(1) mejor	Revisa elemento por elemento
Binaria	Sí	O(log n)	Divide el arreglo a la mitad
Interpolación	Sí	O(log log n) promedio · O(n) peor	Estima la posición con la llave
Salto	Sí	O(√n)	Avanza en bloques de tamaño √n
Hash (repaso)	No	O(1) promedio · O(n) peor	Función hash → índice

Regla práctica: desordenado → secuencial; ordenado → binaria, interpolación o salto; muchas búsquedas sobre los mismos datos → tabla hash.

// búsqueda en arrays

Ahora usted: búsqueda en arrays

Resuelva cada caso y compare su respuesta con un compañero:

A

Secuencial

¿Cuántas comparaciones se hacen para buscar el `40` en `[10, 50, 30, 70, 80, 60, 20, 90, 40]`?

B

Binaria

En `[1…9]`, ¿contra qué elementos del medio se compara para buscar el `3`?

C

Salto

Un array ordenado tiene 100 elementos: ¿qué tamaño de bloque se usa?

D

Ordenado

En `[10, 20, 30, 40]`, ¿cómo evita la secuencial recorrer todo al buscar el `15`?

// pathfinding

Pathfinding: búsqueda de caminos

Camino entre dos puntos de un plano (videojuegos, simulaciones). Depende de:

El destino

¿Es estacionario o móvil?

Los obstáculos

¿Hay obstáculos en el mapa?

El terreno

¿Hay diferentes tipos de terreno en el mapa?

No hay receta única: la elección del algoritmo depende del problema; se suelen combinar varios.

// pathfinding

Pathfinding básico: un paso a la vez

Mover al personaje de A hacia B

```
if (positionX > destinationX)
    positionX--;
else if (positionX < destinationX)
    positionX++;
if (positionY > destinationY)
    positionY--;
else if (positionY < destinationY)
    positionY++;
```

La idea

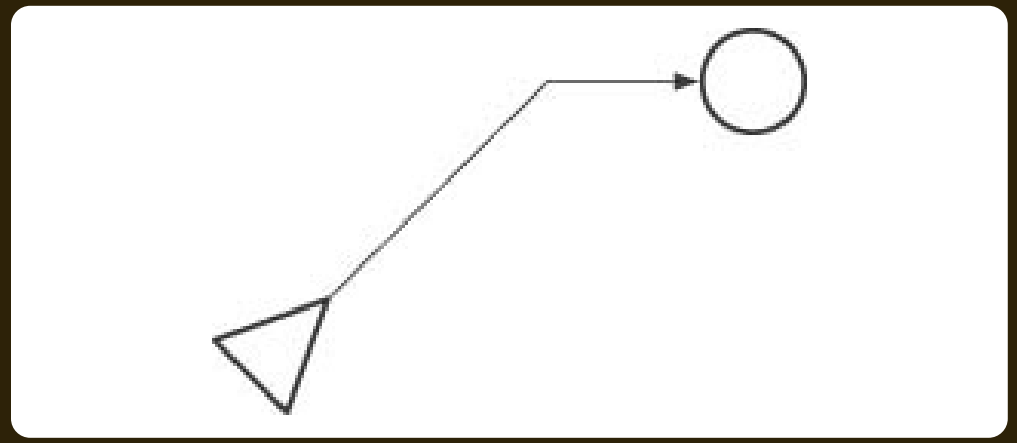
Acercar al personaje un paso en X y otro en Y hacia el destino.

Su límite

No es óptima: no considera obstáculos, terreno ni la distancia real.

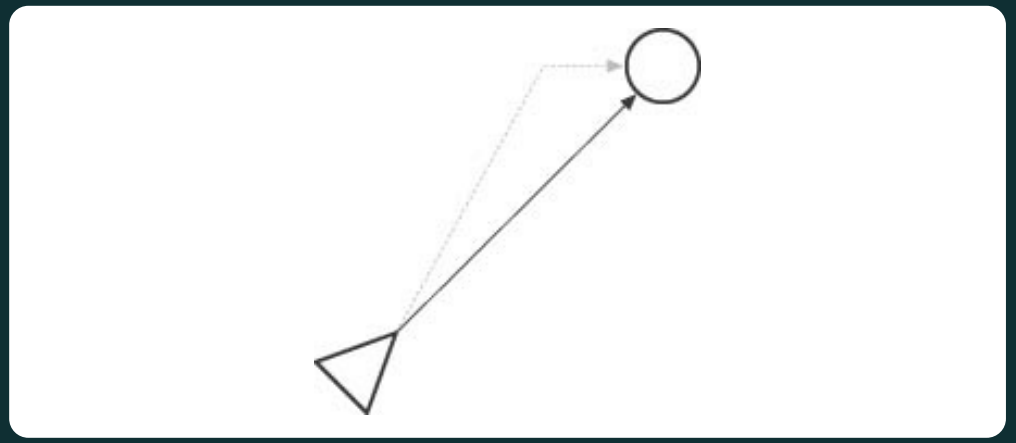
Básico vs. línea de visión

Pathfinding básico



Movimiento poco natural

Línea de visión

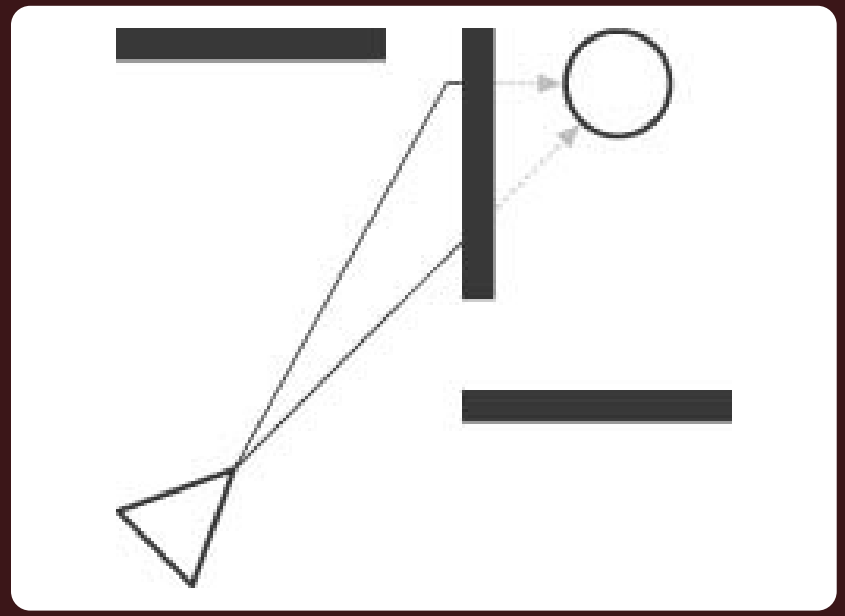


Movimiento esperado

Prefiera la línea de visión (u otros enfoques más sofisticados) cuando sea posible; el método simple es solo una **aproximación**.

Obstáculos: los enfoques simples no bastan

Camino bloqueado



La línea recta choca con la pared

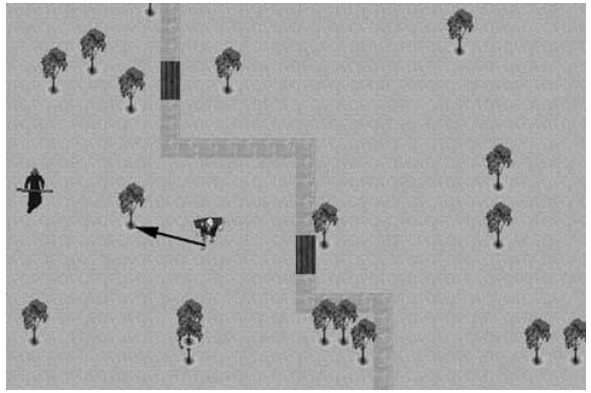
Idea clave

- Pathfinding no se resuelve con **un solo algoritmo**
- Se resuelve con una **combinación** de algoritmos
- Al encontrar obstáculos, el método básico y la línea de visión no son suficientes

// pathfinding

Movimiento aleatorio ante obstáculos

Esquivar al enemigo



Un movimiento aleatorio puede bastar

Pseudocódigo

```
if Player In Line of Sight {
    Follow Straight Path to Player
} else {
    Move in Random Direction
}
```

Aunque parezca poco eficiente, es simple y efectivo con **pocos obstáculos**.

Límite: sirve para esquivar a un enemigo; para **llegar a una meta** entre muchos obstáculos el azar no es fiable y puede no alcanzarla.

// pathfinding

Rodear obstáculos

1

Intentar

El personaje intenta un pathfinding básico hasta el objetivo.

2

Rodear

Si encuentra un obstáculo grande (montaña, agua), pasa a modo de rodeo.

3

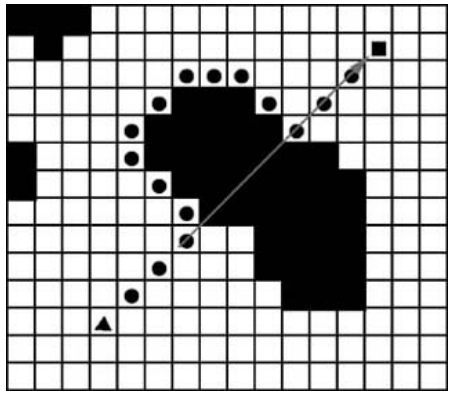
Reintentar

Sigue el borde hasta poder volver al pathfinding básico con éxito.

Difícil: saber cuándo salir del modo de rodeo (una opción: determinar, antes de entrar, por dónde se puede salir). Y aun así, en algunos mapas **puede atascarse**.

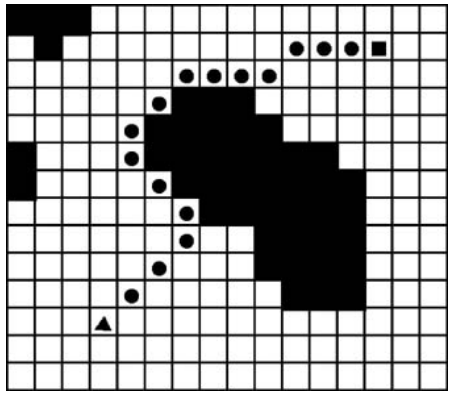
Rodeo con línea de visión en cada paso

Rodeo de obstáculos



Sigue el borde del obstáculo

Con línea de visión

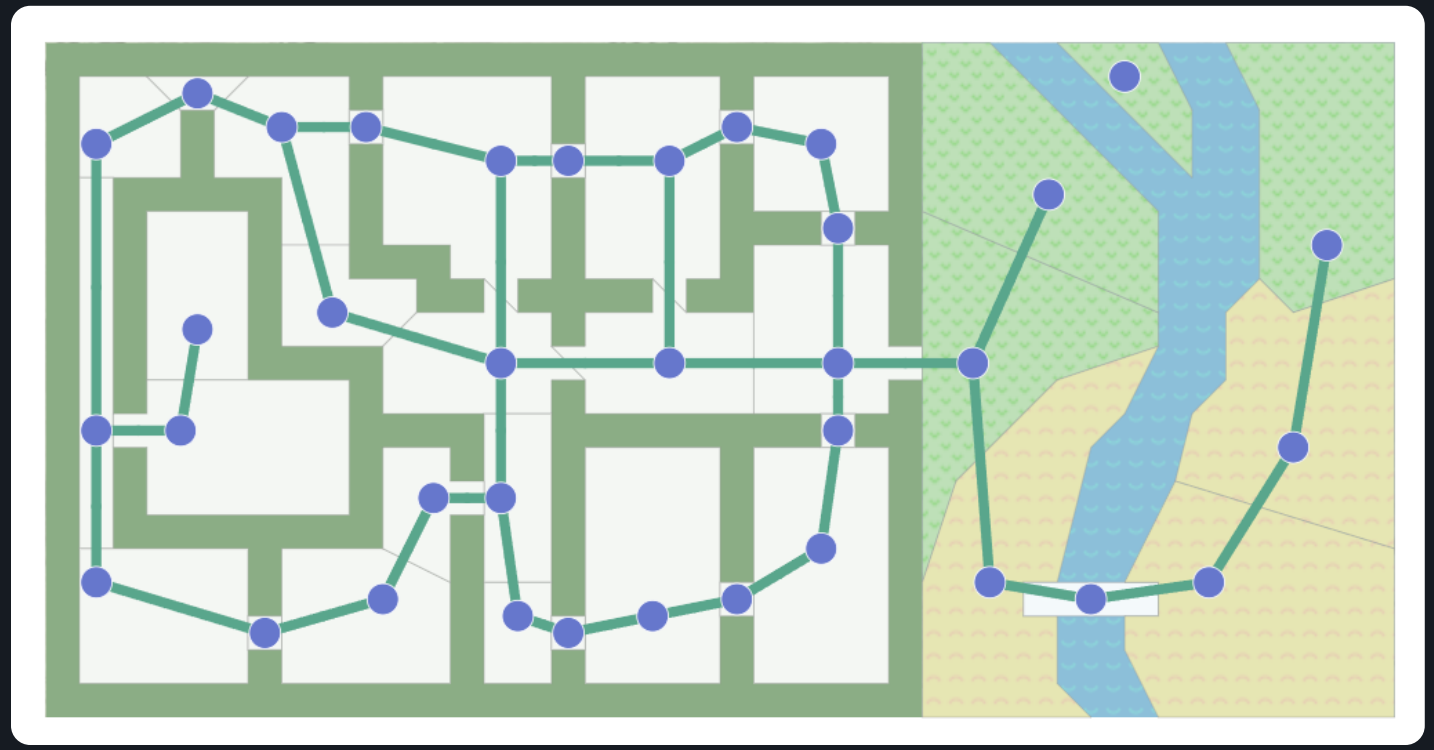


Va recto en cuanto ve el objetivo

En cada paso: si el personaje **ve el objetivo**, avanza en línea recta; si no, sigue rodeando el obstáculo.

El mapa como un grafo

Mapas y sus grafos

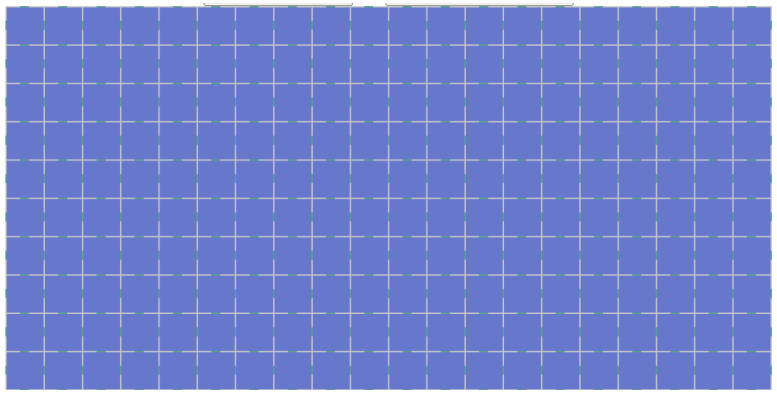


Elementos

- **Nodos:** posiciones en el mapa
- **Aristas:** conexiones entre las posiciones
- **Costo** de cada arista: la distancia o el tiempo que toma recorrerla

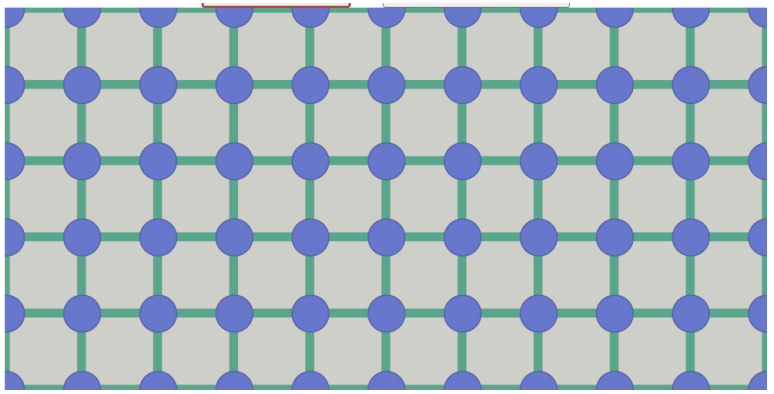
La rejilla (grid) también es un grafo

El mapa como grid



Un mapa modelado como rejilla

El grid es un grafo



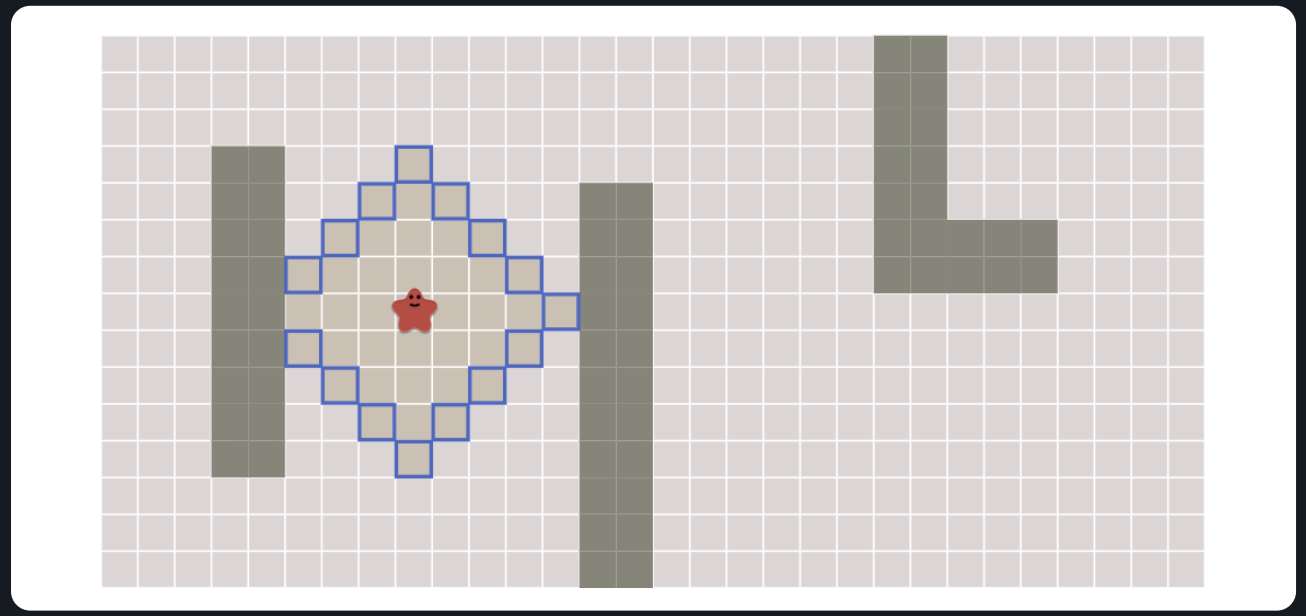
Cada celda es un nodo unido a sus vecinas

Una rejilla es un **tipo especial de grafo**.

// bfs

Breadth-first search (BFS)

Búsqueda en amplitud



Expansión en todas las direcciones

Cómo funciona

- Comienza en un **nodo raíz**
- Explora **todos los vecinos** antes de avanzar a los vecinos de estos
- Se expande **en todas las direcciones**

// bfs

BFS: pseudocódigo

Recordatorio de Estructuras de Datos 1

```
frontier = Queue()
frontier.put(start)
came_from = dict()
came_from[start] = None

while not frontier.empty():
    current = frontier.get()
    if current == goal:
        break
    for next in graph.neighbors(current):
        if next not in came_from:
            frontier.put(next)
            came_from[next] = current
```

frontier

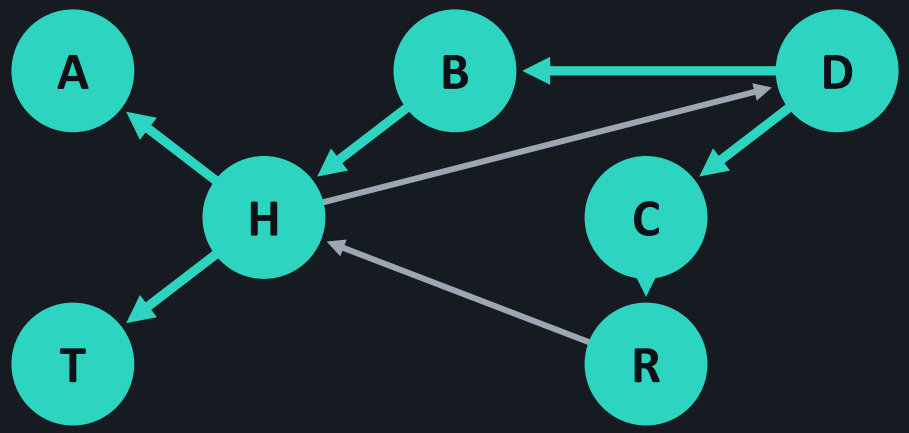
Cola de los nodos que faltan por explorar.

came_from

Guarda desde qué nodo se llegó a cada uno.

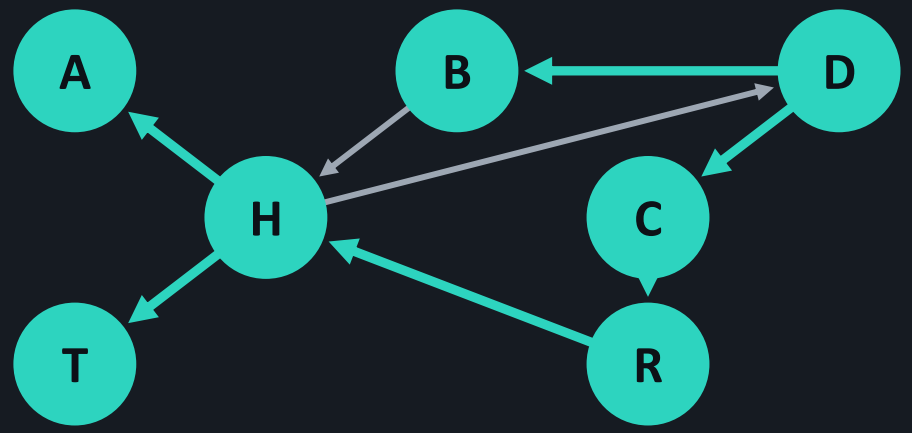
// bfs

BFS vs. DFS: cola contra pila



BFS: cola (FIFO)

Visita por niveles. Orden: **D B C H R A T**



DFS: pila (LIFO)

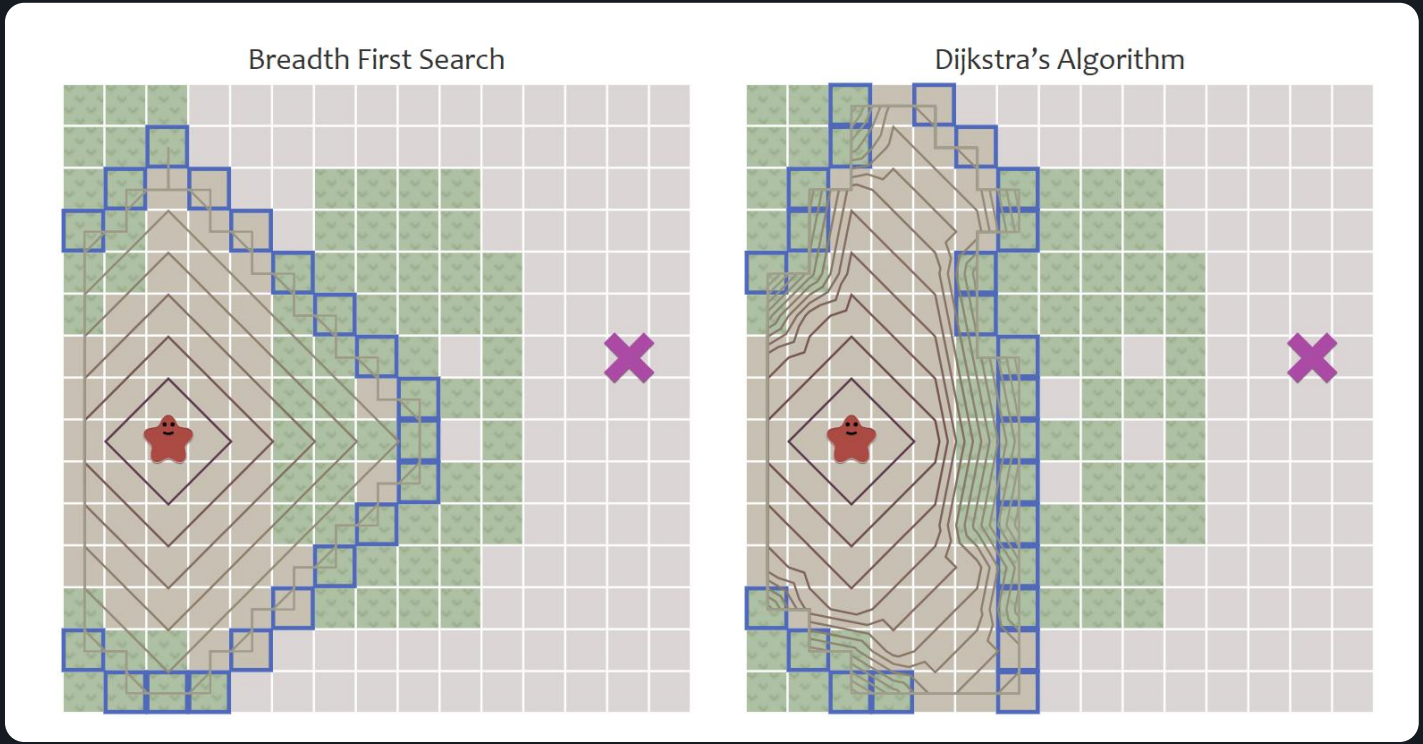
Profundiza antes de volver. Orden: **D C R H T A B**

Ojo: DFS **no** garantiza el camino más corto. Si es recursivo, en un mapa grande puede desbordar la pila: use una pila explícita.

// dijkstra

BFS vs. Dijkstra

BFS y Dijkstra en acción



La diferencia

- **BFS:** todo movimiento cuesta lo mismo
- **Dijkstra:** cada terreno tiene un costo (bosque, agua, desierto…)
- Lleva el **costo acumulado** y usa una **cola de prioridad**

// dijkstra

Dijkstra: pseudocódigo

Inicialización

```
frontier = PriorityQueue()
frontier.put(start, 0)
came_from = {}
cost_so_far = {}
came_from[start] = None
cost_so_far[start] = 0
```

Ciclo principal

```
while not frontier.empty() {
    current = frontier.get()
    if current == goal { break }
    for next in graph.neighbors(current) {
        new_cost = cost_so_far[current] + graph.cost(current, next)
        if next not in cost_so_far
               or new_cost < cost_so_far[next] {
            cost_so_far[next] = new_cost
            priority = new_cost
            frontier.put(next, priority)
            came_from[next] = current
        }
    }
}
```

Qué cambia

Una **cola de prioridad** y el **costo acumulado** deciden el siguiente paso.

// a*

A*: costo real más heurística

Como Dijkstra, pero con una estimación de lo que falta por recorrer:

Costo real

Igual que Dijkstra: lo que ya costó llegar al nodo.

Heurística

Función que **estima** el costo de llegar al destino desde un nodo.

Admisible

Nunca debe **sobreestimar** el costo real.

En la práctica: se usa, con ciertas mejoras, en game engines modernos como Unity.

// a*

A*: una sola línea cambia

Dijkstra

```
cost_so_far[next] = new_cost
priority = new_cost
frontier.put(next, priority)
```

A*

```
cost_so_far[next] = new_cost
priority = new_cost + heuristic(goal, next)
frontier.put(next, priority)
```

Dijkstra

La prioridad es solo el costo acumulado.

A*

La prioridad suma la heurística: guía la búsqueda al destino.

A* es más rápido porque la heurística **guía la búsqueda** hacia el destino.

Peor caso: igual a Dijkstra. Mejor caso: **muchas menos** comparaciones.

De BFS a A*: cambia la prioridad

Algoritmo	Prioridad en la cola	Qué garantiza	Explora
BFS	Orden de llegada (FIFO)	Menos pasos, si costos iguales	Mucho
Dijkstra	`g`: costo acumulado	Camino de menor costo	Mucho
Greedy Best-First	`h`: solo la heurística	No garantiza el más corto	Poco
A*	`g + h`	Camino óptimo si H es admisible	Menos que Dijkstra

> **Lo único que cambia:** el valor de `priority` en `frontier.put(next, priority)`. Todo lo demás es el mismo ciclo.

Heurística: distancia Manhattan

Heurística para una rejilla

```
// Manhattan distance
function heuristic(a, b) {
    return abs(a.x - b.x) + abs(a.y - b.y)
}
```

Se usa en A*

```
priority = new_cost + heuristic(goal, next)
```

Manhattan

`|dx| + |dy|`: suma de las diferencias en x y en y.

Euclidiana

`sqrt(dx² + dy²)`: distancia en línea recta entre los dos puntos.

Ojo: según el escenario, la heurística se puede **precalcular** y guardar en una tabla de búsqueda.

// a*

A*: el mismo algoritmo, con otros nombres

En la literatura	En el pseudocódigo	Significa
G	`cost_so_far[next]`	Costo acumulado desde el inicio
H	`heuristic(goal, next)`	Estimación de lo que falta hasta la meta
F = G + H	`priority`	Decide el orden: se expande el menor F
Lista abierta	`frontier`	Nodos descubiertos por explorar
Lista cerrada	salieron de `frontier`	Nodos ya expandidos
Re-parent	`new_cost < cost_so_far[next]`	Si G mejora, se cambia el padre

El ciclo: se expande el nodo de **menor F**, se descubren sus vecinos y se repite hasta sacar la meta de la lista abierta.

A* paso a paso: el mapa y la heurística H

1 **9**	2 **8**	3 **7**	4 **6**	5 **5**	6 **6**
7 **8**	8 **7**	9 **6**	10 **5**	11 **4**	12 **5**
13 **7**	14 **Inicio** **6**	15 **5**	16 Muro	17 **3**	18 **4**
19 **6**	20 **5**	21 **4**	22 **3**	23 **2**	24 **3**
25 **5**	26 Muro	27 **3**	28 Muro	29 **1**	30 **2**
31 **4**	32 **3**	33 **2**	34 Muro	35 **Meta** **0**	36 **1**
37 **5**	38 **4**	39 **3**	40 **2**	41 **1**	42 **2**

- Movimiento en **4 direcciones**, con costo **1** por paso.
- **H** = distancia Manhattan hasta la meta: filas + columnas que faltan, sin contar los muros.
- Ejemplo: del nodo 14 al 35, H = 3 + 3 = **6**.

> En cada celda: el número del nodo (arriba) y su **H** (centro).

A* paso a paso: primera expansión

1 9	2 8	3 7	4 6	5 5	6 6
7 8	8 **1+7=8**	9 6	10 5	11 4	12 5
13 **1+7=8**	14 **Inicio** **0+6=6**	15 **1+5=6**	16 Muro	17 3	18 4
19 6	20 **1+5=6**	21 4	22 3	23 2	24 3
25 5	26 Muro	27 3	28 Muro	29 1	30 2
31 4	32 3	33 2	34 Muro	35 **Meta** **0**	36 1
37 5	38 4	39 3	40 2	41 1	42 2

Cerrado Abierto Siguiente

Lista abierta (G + H = F)

Nodo	G	H	F	Padre
8	1	7	8	14
13	1	7	8	14
15	**1**	**5**	**6**	**14**
20	1	5	6	14

Se expande el de **menor F**: el 15 y el 20 empatan (F = 6); gana el de menor número.

A* paso a paso: el resultado

A*: 7 nodos expandidos



Dijkstra: 30 nodos expandidos

16	9	15	21	27	
10	4	7	13	19	28
5	1	2		23	
8	3	6	11	17	24
14		12		22	
20	25	18		30	
29		26			

Inicio · Meta · Camino de A* · Expandido · Descubierto, sin expandir

Mismo costo (6): A* expandió **7** nodos; Dijkstra, **30** de las 38 celdas libres. El número es el orden de expansión.

// práctica

Ahora usted: ¿qué algoritmo uso?

Elija la técnica de pathfinding más adecuada para cada caso:

A

Mapa abierto

Un personaje debe ir a un punto, sin obstáculos ni terrenos distintos.

B

Terrenos con costo

Bosque, agua y desierto afectan la velocidad. ¿Qué algoritmo y qué estructura usa?

C

Rejilla grande

Un solo destino conocido en una rejilla. ¿Cómo mejora A* a Dijkstra?

// cierre

Resumen

- **Secuencial:** `O(n)`; la única opción si el array está desordenado.
- **Binaria, interpolación y salto:** requieren arrays ordenados.
- **Interpolación:** `O(log log n)` en promedio; **salto:** `O(sqrt(n))`.
- **Pathfinding:** no hay solución única; se combinan básico, línea de visión, aleatorio y rodeo.
- **Grafos: BFS** (costos iguales), **Dijkstra** (costos distintos) y A* (`F = G + H`).

Referencias: GeeksforGeeks, «Jump Search»; OpenGenus, «Time complexity of Interpolation Search»; Wikipedia, «Pathfinding»; Red Blob Games, «Pathfinding on Grids» (redblobgames.com/pathfinding/a-star/introduction.html); Bourg, D. M. (2004). AI for Game Developers. O'Reilly Media. ISBN 978-0-596-00555-2.